

birdclef-2026

5位解法: 多様性とバグ - この2つこそがすべて (Diversity and Bug - Both Are All You Need)

Rank	5
Team	Jiacheng Ma
Author	马家☒
Topic ID	704602
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704602>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

原文（Kaggle Discussion）：<https://www.kaggle.com/competitions/birdclef-2026/discussion/704602>

まず、この素晴らしいイベントを開催してくれたスポンサーとKaggleに感謝します。高品質なデータと公平な環境のおかげで、このコンペティションは挑戦的でやりがいのあるものになりました。

知見やノートブック、ディスカッションを共有してくれたすべての参加者に心から感謝します。このコミュニティの協力的な精神には本当に刺激を受けました。皆さんから多くを学びましたし、そうした交流がなければこの解法は存在しませんでした。

これは私にとって初めての金メダルであり、いまだに実感が湧きません。トップに到達するには本物の運が必要でした。恵まれたバリデーション分割、幸運なアンサンブルの選択、そして私に有利に働いたタイミング。また、多くの優れたコンペティターがprivateのsubmissionでより高いスコアを達成しながら、それを最終提出として選ばなかったことも認識しています。もし彼らがそれを選んでいれば、リーダーボードはまったく違ったものになっていたでしょう。

最後に、私はただ道中ずっと幸運だっただけです。時にはすべてがうまくかみ合うことがあり、これはそんな稀な瞬間の一つでした。忘れられないコンペティションをありがとうございました。

Data

コンペの **focal + soundscapes**。私のアーキテクチャでは、コンペのfocalデータとsoundscapesデータを単純に連結するだけで最良の結果が得られました。かつてCVとLBの相関を見つけようとしたことが、最終的には失敗しました。私はfocalデータの一部から分割したバリデーションセットのAUCとlossにのみ基づいてモデルを選定し、ラベル付きのsoundscapesはすべて学習に使用しました。

Augmentation

- **Mixup**: max-lambda戦略を用いた音声レベルのミックス ($\lambda = \max(\beta, 1-\beta)$ 、ここで $\beta \sim \text{Beta}(\alpha, \alpha)$)。元のサンプルがミックスの中で支配的になる ($\lambda \geq 0.5$) ことを保証します。
- **Filter_aug**: 周波数帯域全体にわたってランダムな区分線形ゲインを適用し、スペクトルのカラーレーション（着色）をシミュレートします。
- **Wave-noisy**: ゲインのジッター、加法ノイズ、ランダムシフト。
- **SpecAugment** (time maskingのみ) : メルスペクトログラム内の連続したタイムステップをランダムにマスクします。

Model Families

すべてのcnnモデルは公開ベースライン [BirdCLEF+2026: HGNetV2-B0 Baseline \[Training\]](#) (@ttahara に感謝) に基づいており、元のSEDヘッドを別の設計に置き換えています。そして最終的にはすべてperch-distilledに基づいています。

backbone	pretrain	lms_shape	duration	loss	upsampling	perch-distilled	self-distilled	pseudo	fold	seed A/B
hgnetv2b0	-	128, 313	5s	bce	√	√	×	×	0	520/3407
efficientnetv2s	bird-clef-2025-all-pretrained-models	128, 313	5s	bce	×	√	2-iter	1-iter	0	1086/42
efficientnetv2s	bird-clef-2025-all-pretrained-models	128, 626	10s	bce	×	√	×	×	0	1086/1
efficientnetb3	-	384, 384	5s	bce	×	√	×	1-iter	0	1086/888

私の各シングルモデルについて、ハイパーパラメータのチューニングを別にとすると、最も大きなゲインをもたらしたのは次の要素でした: **蒸留 (Distillation)** (50%)、**各種データ拡張** (30%)、**モデルアーキテクチャ** (20%)。私の最良のシングルモデルについて、その詳細の一部は以下の通りです:

Improve	Public Score
Baseline	0.88-0.895
+ Data Aug	0.895-0.905
+ Perch-Distilled	0.905-0.915
+ Pretrained	0.915-0.925
+ Self-Distilled	0.925-0.928
+ Pseudo iter 1	0.928-0.935
+ TTA	+0.005-0.008
Post-Processing	+0.005-0.01

昨年のコンペティションでの優れた解法である [2nd Place. Journey Down the Rabbit Hole of Pseudo Labels](#) と [5th place solution: Self-Distillation is All You Need](#) に特別な感謝を捧げます。これらは私のアンサンブルおよび疑似ラベリング (pseudo-labeling) 戦略に大きな影響を与えました。

Post-Processing

- **TTA:** デュアルウィンドウ推論 (通常 + 2.5sシフト)。

- **Smoothing:** 隣接フレームを用いた [0.1, 0.8, 0.1] のウィンドウでスムージングを適用（[昨年の5位解法](#)）。
- **File Peak Scale:** 各音声ファイル内の予測を、上位k個のウィンドウ確率の平均でスケーリングします（[昨年の2位解法](#)）。

Final Ensemble

- **CNNアンサンブル:** SEDヘッド付きの hgnetv2-b0 + efficientnetv2-s + efficientnet-b3。推論では clipwiseのみを使用。
- **系列モデル (Sequential model) :** 公開モデルに基づく ProtoSSM（共有してくれた方々に感謝）
- **Distilled SED Baseline** (@tuckerarrants に感謝)

```
Final = 0.6 * (0.25 * hgnet + 0.40 * effv2_5s + 0.25 * effv2_10s + 0.10 * effb3) + 0.4 * (0.6 * proto + 0.4 * sed)
```

多様性を最大化するため、私はMelパラメータを可能な限り最小化し、シングルモデルの推論を高速化して、多くのモデルをアンサンブルし分散を減らせるようにしました。だから私が言いたいのは **多様性こそがすべて (Diversity Is All You Need)** ！ということです。学んだ教訓が一つあります: 同じ後処理を複数のモデルに適用すると、実際にはLBが悪化するのです。そこで最終submissionでは、代わりに **異なる位置で異なる後処理** を適用しました。

Bug

ここまで、上で述べてきたように、私の取り組みはかなりオーソドックスなものであり、擬似ラベリングの部分でさえ特別うまく実行できたわけではありませんでした。コンペ終了前夜、私はまだ自分のモデルが大きなスコア変動や下落に見舞われるかもしれないと心配していました。というのも、私のデータ処理はあまり頑健ではなく、データの多様性そのものも強くなかったからです。だからこそ、翌日にprivateリーダーボードで自分のモデルがどれほど良い成績を収めたかを見たとき、自分自身少し驚きました。それは驚くほどprivateリーダーボードによくフィットしていたのです。

Submission and Description	Private Score ^①	Public Score ^①	Selected
 <u>PayOff - Version 1</u> Succeeded · 2d ago · Paid Off	0.95824	0.95715	☒
 <u>ensemble2post - Version 6</u> Succeeded · 2d ago	0.95713	0.95712	☐
 <u>ensemble2post - Version 5</u> Succeeded · 2d ago	0.95765	0.95710	☐
 <u>ensemble2post - Version 4</u> Succeeded · 2d ago	0.95715	0.95714	☐
 <u>ensemble2post - Version 3</u> Succeeded · 2d ago	0.95715	0.95714	☒
 <u>ensemble2post - Version 2</u> Succeeded · 3d ago	0.95705	0.95701	☐

そこで私は具体的な理由を探り始め、最終的に「バグ」と呼んでいる処理上のディテールを発見しました。データ拡張において、一般的なFrequencyFilterAugは本質的に $\text{dB} + \text{noise}$ (dBドメインでの加算/減算) ですが、私のアプローチは $\text{dB} \times \text{linear_gain}$ (dBドメインでの乗算) でした。物理的な音声処理の観点から言えば、Log-Melを乗算するとメルスペクトログラムの物理的な意味が完全に破壊されます。元々-80dBの静かな周波数帯域が、2倍 (6dBのゲインに相当) されると-160dBになり、これは物理的にはあり得ません。それにもかかわらず、この「dBドメインでのクレイジーなスケールリング」の乗算操作は予想外に優れた汎化性能をもたらしました。私の部分的なテストを通じて、この手法は私のモデルに 0.01+ という大きな改善をもたらしました。以下の「God Trick」に似ています:

```
def freq_filt_aug(features, db_range=(-6, 6), n_band=(3, 6), min_bw=6):
    B, n_freq, _ = features.shape
    n_bands = torch.randint(n_band[0], n_band[1], (1,)).item()
    if n_bands <= 1:
        return features
    while n_freq - n_bands * min_bw + 1 < 0:
        min_bw -= 1
    bndry = torch.sort(
        torch.randint(0, n_freq - n_bands * min_bw + 1, (n_bands - 1,))
    )[0] + torch.arange(1, n_bands) * min_bw
    bndry = torch.cat([torch.tensor([0]), bndry, torch.tensor([n_freq])])
    factors = torch.rand((B, n_bands + 1), device=features.device) * (db_range[1] - db_range[0]) + db_range[0]
    freq_filt = torch.ones((B, n_freq, 1), device=features.device)
    for i in range(n_bands):
        l, r = int(bndry[i].item()), int(bndry[i + 1].item())
        for j in range(B):
            freq_filt[j, l:r, :] = torch.linspace(
                factors[j, i].item(), factors[j, i + 1].item(), r - l,
                device=features.device,
            ).unsqueeze(-1)
    return features * (10 ** (freq_filt / 10))
```

Closing

私はまだ解法投稿を書くのに慣れていないので、不十分な点があればご容赦ください。どんな提案やアドバイスも大歓迎です。私はいつだってこの素晴らしいコミュニティから学びたいと思っています。改めて、皆さんに感謝します。このコミュニティは本当に特別です。😊

Resources

- 推論カーネル: [5th-solution notebook](#)
- 最終推論用の重み: [birdclef2026-5th-final-cnn-models](#)
- Githubリポジトリ: [BirdCLEF2026-5th-solution](#)